

Figure 3: (a) shows the transition of single trainable σ for VGG-16 architecture on the three datasets. (b) shows the Layerwise mean σ value for VGG-16 ProbAct layers trained on CIFAR10 dataset. (c) shows the test accuracy comparison between ReLU with noisy input and ProbAct, while (d) shows the test accuracy comparison of ReLU, ReLU with noisy activation and ProbAct on CIFAR-10 dataset.

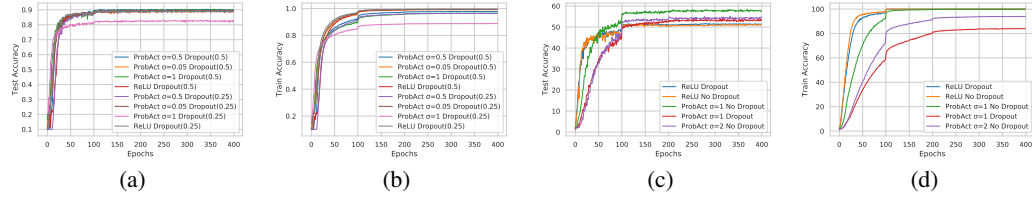


Figure 4: Training element-wise μ with a high σ value acts an inbuilt regularizer that generalizes better on data, preventing overfitting. We evaluate the network generalization capabilities based on the train and test performance of the network. (a) and (c) shows the test accuracy comparison between ReLU and ProbAct layer with/without dropout layers on CIFAR-10 and CIFAR-100 datasets respectively, while (b) and (d) shows the train accuracy comparison of the same. Network with ProbAct layers achieves better test results and a lower train test accuracy difference, showing the built-in regularization nature of ProbAct.

Figure 2 shows the frequency distribution for the bounded element-wise trained k values after 400 epochs. We observed two peak values for every distribution across all three image datasets. We assume that the derivative of a sigmoid function becomes 0 at both the boundaries of the function. The points in the left peak lie in the lower boundary of σ (0 in our case) making ProbAct behave as ReLU. Right peak points lie in the upper boundary of the sigmoid (2 in our case) and take σ as 2. The values in between the peaks signify other σ values.

In the case of both CIFAR datasets, the distribution of parameter k in the last layer is quite narrow and concentrated in the negative domain. As shown in Figure 3 (b), the σ values becomes 0, which indicates that ProbAct conducts the ReLU-like operation.

4.3 Augmentation by Activation

We add ProbAct to the first layer of the VGG network keeping ReLU as the activation function for all the other layers. We show that perturbation induced by ProbAct in the first layer behaves as augmentation added in the activation function; improving the overall network generalization ability. ProbAct can be thought of as a way to add an adaptable and trainable perturbation enhancing the generalization capability of the network. These added perturbations are different than just adding noise to either the activation or to the inputs. Figure 3 (c) draws a comparison between ProbAct in the first layer with noisy input (in our experiments, Gaussian noise was added to the inputs sampled from a distribution $\mathcal{N}(0, 1)$), while in Figure 3 (d), Gaussian noise is added to the first activation layer. ProbAct with learnable variance outperforms both the standard ReLU with noisy inputs and standard ReLU with noisy activations. Our proposed method adopts stochastic noise into the activation function in a controlled manner. To the best of our knowledge, our proposed method is the first approach that adopts learnable stochastic noise into activation function.

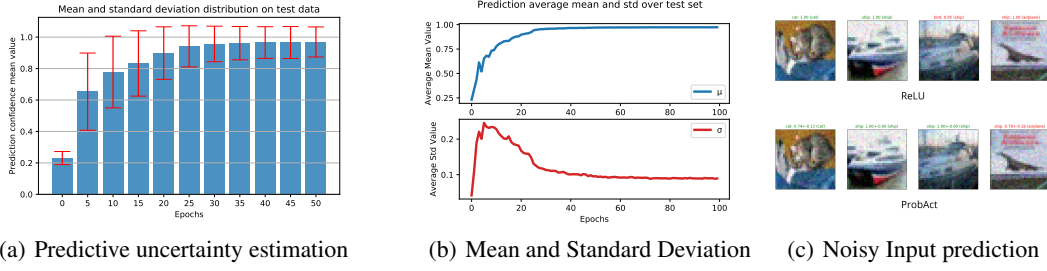


Figure 5: (a) shows the predictive mean (blue bar) and the standard deviation (error margins in red). With more data, the model uncertainty decreases (clearly seen with increasing mean and decreasing variance). (b) demonstrates the increase in predictive confidence mean is directly proportional to the decrease in the variance. (c) shows that for ProbAct prevents over confident decisions on noisy inputs.

4.4 Uncertainty Estimation

ProbAct adds stochastic behavior to the neural network and it can be defined as:

$$p(y | x, \theta) = \int p(y | x, \theta, \epsilon) p(\epsilon) d\epsilon = \mathbb{E}_{p(\epsilon)} f(x, \theta, \epsilon) \quad (7)$$

where, y : predicted output; x : input; θ : model parameters; ϵ : noise; X : mini batch data.

ProbAct learns a Gaussian distribution for every element over time. Each sample from this distribution propagates a new output to the next layers, eventually allowing ProbAct to learn an infinite ensemble of networks. With these ensemble networks, the predictive mean and variance can be estimated by calculating the first and second-order moments. We also observe a decrease in the predictive uncertainty as the network becomes more confident with its decision as shown in Figure 5 (a) and (b) for CIFAR-10. We also show that this predictive uncertainty helps in preventing overconfident predictive decisions for noisy data. Figure 5 (c) shows that for noisy inputs, instead of providing a fixed confident predictive score for a misclassification, ProbAct provides a range of low predictive confidence score, demonstrating its non-confident behaviour.

4.5 Reduced Data

The training sample size was reduced to 50% and 25% of the original data size for CIFAR-10 and CIFAR-100 dataset. We maintained the class distribution by randomly choosing 25% and 50% images for each class. The process was repeated three times to create three randomly chosen datasets. We run our experiments on all three datasets and average the results.

Table 2 shows the test accuracy for ReLU and ProbAct with Element-wise Trainable σ (bound) on 25% and 50% data size. We achieve 3% average increase in test accuracy when the data size was halved and 2.5% increase when it was further halved. The higher test accuracy of ProbAct shows the applications of ProbAct in real-life use cases when the training data size is small.

5 Conclusion

In this paper, we introduced a novel probabilistic activation function, ProbAct, that adds perturbation in every activation map, allowing better network generalization capabilities. Through experiments we

Table 2: Test Accuracy (%) comparison between ReLU and ProbAct on reduced subsets of CIFAR-10 and CIFAR-100 (50% and 25% of original dataset). The test accuracy(%) indicates the average of three sets of testing.

Activation function	CIFAR-10 (50%)	CIFAR-100 (50%)	CIFAR-10 (25%)	CIFAR-100 (25%)
ReLU	82.74	42.36	75.62	30.42
ProbAct	84.73	46.11	79.02	31.67

verified that the stochastic perturbation prevents the network from memorizing the training samples, resulting in evenly optimized network weights and a more robust network with a lower generalization error. Furthermore, we confirmed that the augmentation-like operation in ProbAct is effective for classification tasks even when the number of data points is very less. Finally, we demonstrate that ProbAct is very robust to noisy input data and provides an estimate for predictive uncertainty.

Acknowledgement

We would like to acknowledge the members of Uchida Lab, Fukuoka and Mr. Felix Laumann for their invaluable discussions.

6 Appendix

The VGG-16 architecture used in the experiments is defined as follows:

VGG16 : $[64, 64, M, 128, 128, M, 256, 256, 256, M, 512, 512, 512, M, 512, 512, 512, M, C]$

where, numbers 64, 128 and 256 represents the filters of *Convolution layer* which is followed by a Batch Normalization layer, followed by an activation function. M represents the *Max Pooling layer* and C represents the *Linear classification layer* of dimension (512, number of classes).

Other hyper-parameters settings include:

Table 3: Hyper-parameters for the experiments

Hyper-parameter	Value
Convolution Kernel Size	3
Convolution layer Padding	1
Max-Pooling Kernel Size	2
Max-Pooling Stride	2
Optimizer	Adam
Batch Size	256
Fixed σ values	[0.05, 0.1, 0.25, 0.5, 1, 2]
Learning Rate	0.01 (Dropped 1/10 after every 100 epochs)
Number of Epochs	400
Image Resolution	32×32
Single trainable σ Initializer	Zero
Element-wise trainable σ Initializer	Xavier initialization

6.1 Datasets

6.1.1 Image Datasets

CIFAR-10 Dataset The CIFAR-10 dataset consists of 60,000 images with 10 classes, with 6,000 images per class, each image 32 by 32 pixels. The dataset is split into 50,000 training images and 10,000 test images.

CIFAR-100 Dataset CIFAR-100 dataset has 100 classes containing 600 images per class. There are 500 training images and 100 test images per class. The resolution of the images is also 32 by 32 pixels.

STL-10 Dataset STL-10 dataset has 500 images per class with 10 classes and 100 test images per class. The images are 96 by 96 pixels per image.